In-Class Coding Activity: Building and Evaluating a Neural Network

Topic: Neural Networks, Model Training, Hyperparameter Tuning, and Model Evaluation
Dataset: Wine Recognition Dataset (`sklearn.datasets.load_wine`)
Working arrangement: Groups of 2–3 students

Learning objectives

By the end of this activity, you should be able to:

1. Load a dataset and identify its input features and target variable.
2. Prepare data for training a neural network.
3. Design a flexible Artificial Neural Network (ANN).
4. Train a neural network and monitor training and validation loss.
5. Explain the difference between model parameters and hyperparameters.
6. Perform hyperparameter tuning using grid search.
7. Select the best model using validation performance.
8. Evaluate a multiclass classification model using a confusion matrix, precision, recall, and F1-score.

Part 1: Load and explore the dataset

We will use the Wine Recognition Dataset from Scikit-learn.

The dataset contains:

- 178 wine samples.
- 13 numerical input features.
- 3 target classes.

Run the following code:

```
import warnings
warnings.filterwarnings("ignore", category=Warning)
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import torch
import torch.nn as nn

from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
    accuracy_score
)

wineData = load_wine()

X = pd.DataFrame(
    wineData.data,
    columns=wineData.feature_names
)

y = pd.Series(wineData.target)

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

print(X.head())
print(y.value_counts().sort_index())
```

Questions:

1. How many observations are in the dataset?
2. How many input features does each observation have?

3. How many classes are there?
4. Is this a regression or classification problem? Why?

Part 2: Prepare the data

Split the dataset into training, validation, and testing sets.

We will use:

- 60% training data.
- 20% validation data.
- 20% testing data.

```
# First: separate training data from the remaining data
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.40,
    random_state=42,
    stratify=y
)

# Second: split the remaining data into validation and test sets
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

print("Training:", X_train.shape)
print("Validation:", X_val.shape)
print("Testing:", X_test.shape)
```

Standardize the features

The features have different measurement scales. For example, some features may have values in the hundreds, while others may have values below 1.

Standardization helps the neural network learn more effectively.

```
scaler = StandardScaler()

# Fit only on training data
X_train = scaler.fit_transform(X_train)

# Transform validation and test data using the same scaler
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

Convert the data into PyTorch tensors:

```
X_train = torch.tensor(X_train, dtype=torch.float32)
X_val = torch.tensor(X_val, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train.values, dtype=torch.long)
y_val = torch.tensor(y_val.values, dtype=torch.long)
y_test = torch.tensor(y_test.values, dtype=torch.long)
```

Questions:

1. Why do we standardize the features?
2. Why do we fit the scaler using only the training data?
3. What is the purpose of the validation dataset?
4. Why should we keep the test dataset separate?

Part 3: Design a flexible ANN

Complete the following ANN architecture.

Your model should allow you to experiment with different numbers of hidden layers and neurons. Demo code is given below.

```
class RegressionNN(nn.Module):
    def __init__(self, hidden_sizes):
        super().__init__()
```

```
        layers = []
        input_size = 1 # set the input size according to your dataset
        output_size = 1 # set according to the dataset

        for hidden_size in hidden_sizes:
            layers.append(
                nn.Linear(input_size, hidden_size)
            )
            layers.append(nn.Tanh())

            input_size = hidden_size

        # One output neuron because this is regression
        layers.append(nn.Linear(input_size, output_size))

        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

Questions:

1. Why do we need an activation function?
2. How to calculate the number of input neurons in the first layer?

Part 4: Train the model

Part 5: Hyperparameter tuning using grid search

```
search_space = [

    # Architecture, learning rate, weight decay

    ([2],       0.01, 0.0),

    ([8],       0.01, 0.0),
```

```
    ([16],      0.01, 0.0),
    ([64],      0.01, 0.0),
    ([16, 16],  0.01, 0.0),
    ([64, 64],  0.01, 0.0),
    ([16],      0.001, 0.0),
    ([16],      0.01, 0.01),
]

results = []
candidate_models = []

for hidden_sizes, lr, wd in search_space:
    torch.manual_seed(42)

    model = RegressionNN(hidden_sizes)

    train_losses, val_losses = train_model(
        model,
        X_train, y_train,
        X_val, y_val,
        epochs=500,
        learning_rate=lr,
```

```
        weight_decay=wd
    )

    # Use the final validation loss for this simple demonstration
    results.append({
        "hidden_layers": str(hidden_sizes),
        "learning_rate": lr,
        "weight_decay": wd,
        "train_mse": train_losses[-1],
        "validation_mse": val_losses[-1]
    })

    candidate_models.append(model)

results_df = pd.DataFrame(results)
results_df = results_df.sort_values("validation_mse")
```

Questions:

1. Which configuration has the lowest validation loss?
2. Does the model with the lowest training loss also have the lowest validation loss?
3. How does changing the number of hidden neurons affect model performance?
4. What is the role of weight decay?
5. Which configuration would you select, and why?

Part 6: Evaluate the best model using the test dataset

After selecting the best configuration, evaluate it using the test dataset.

Do not use the test dataset to select or tune the model.

```
# Retrieve the best model
best_model = candidate_models[
    results_df.index[0]
]

best_model.eval()

with torch.no_grad():

    test_predictions = best_model(X_test)

    # Convert output logits into predicted class labels
    y_pred = torch.argmax(
        test_predictions,
        dim=1
    )

# Calculate test accuracy
test_accuracy = accuracy_score(
    y_test.numpy(),
    y_pred.numpy()
)

print("Test Accuracy:", test_accuracy)
```

Generate the confusion matrix

```
cm = confusion_matrix(
    y_test.numpy(),
    y_pred.numpy(),
    labels=[0, 1, 2]
)
```

```
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=wineData.target_names
)

disp.plot(
    cmap="Blues",
    values_format="d"
)

plt.title("Confusion Matrix - Test Dataset")
plt.show()
```

Generate the classification report

```
print(
    classification_report(
        y_test.numpy(),
        y_pred.numpy(),
        target_names=wineData.target_names
    )
)
```